МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №1
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск с возвратом

Студент гр. 4343 _____ Васильев А. С.
Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург
2026

Цель работы

Изучение итеративного алгоритма поиска с возвратом и реализация алгоритма разбиения квадрата на минимальное число меньших квадратов.

Вариант

Работа выполнена в соответствии с вариантом 2и. Итеративный бэктрекинг. Исследование времени выполнения от размера квадрата.

Задание

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до N−1, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N. Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков (Рисунок 1).

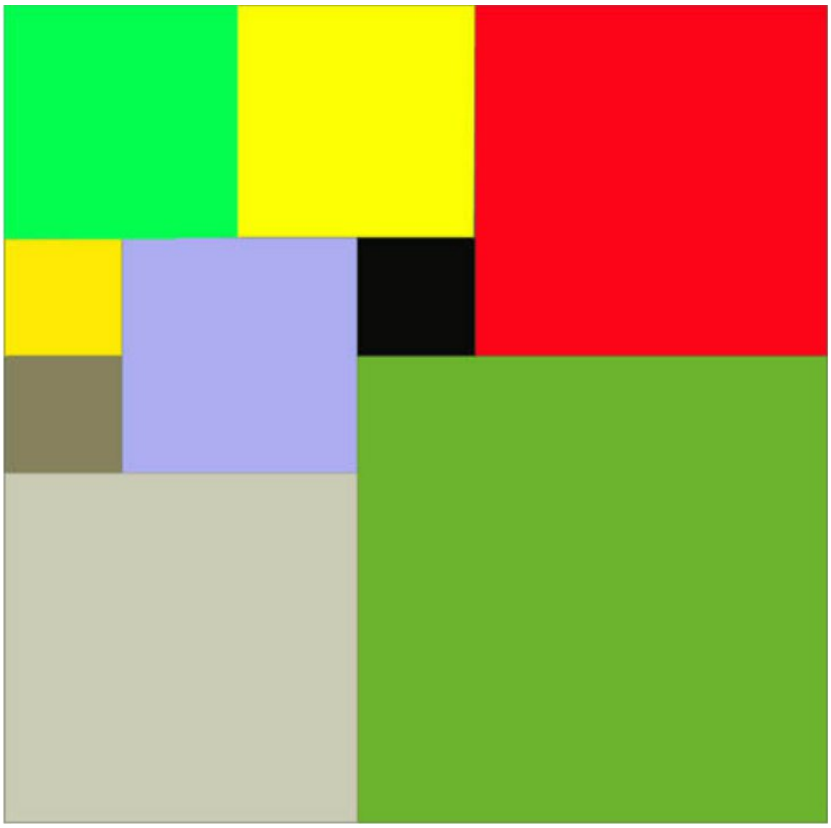

Рисунок 1 – Столешница размера 7×7

Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы - одно целое число N ($2 \le N \le 20$).

Выходные данные

Одно число K, задающее минимальное количество обрезков(квадратов), из которых можно построить

столешницу(квадрат) заданного размера N. Далее должны идти K строк, каждая из которых должна содержать три целых числа x,y и w, задающие координаты левого верхнего угла ($1 \le x,y \le N$) и длину стороны соответствующего обрезка(квадрата).

Теоретический минимум

Бэктрекинг — это метод перебора, при котором решение строится по шагам. Если на каком-то шаге становится ясно, что получить правильный или лучший ответ уже нельзя, алгоритм возвращается назад и пробует другой вариант. Такой подход позволяет не перебирать все варианты полностью, а отсекать бесперспективные ветви раньше.

Обычно бэктрекинг используют в задачах, где нужно выбрать один из многих вариантов и при этом соблюдать ограничения. На каждом шаге строится частичное решение, затем проверяется, допустимо ли оно. Если ограничение нарушено, дальнейший поиск по этой ветви прекращается.

Эффективность бэктрекинга сильно зависит от оптимизаций. Чем раньше удаётся обнаружить бесперспективную ветвь, тем меньше состояний придётся рассмотреть. Поэтому в таких алгоритмах часто применяют отсечения, жадные начальные оценки, уменьшение числа симметричных вариантов и более удобные структуры хранения состояния.

Описание алгоритма

Для решения задачи использован итеративный бэктрекинг. Поле $N \times N$ хранится в одномерном массиве, где каждая клетка содержит признак

занятости. Частичное решение представляется вектором уже поставленных квадратов, а лучшее найденное решение хранится отдельно.

Для реализации поиска без рекурсии используется стек кадров. Каждый кадр содержит координаты первой свободной клетки, текущий размер квадрата для перебора, строку начала поиска следующей свободной клетки и информацию о квадрате, которым был выполнен переход на данный уровень.

Сначала строится жадное допустимое решение, которое задаёт начальную верхнюю границу. Для чётных N используется отдельный специальный случай: квадрат сразу делится на 4 равных части. Для нечётных N первый квадрат фиксируется в левом верхнем углу, а его размер перебирается от N - 1 до (N + 1) / 2.

В основном цикле алгоритм выбирает очередной размер квадрата, проверяет возможность его постановки, выполняет постановку и применяет отсечения. Если ветвь бесперспективна, происходит откат. Если найдено полное покрытие с меньшим числом квадратов, лучшее решение обновляется. По завершении работы алгоритма получается минимальное разбиение квадрата.

Оценка сложности

Временная сложность алгоритма в худшем случае является экспоненциальной, так как задача решается методом перебора с возвратом, и число рассматриваемых состояний быстро растёт при увеличении размера квадрата. Дополнительные оптимизации уменьшают число реально просматриваемых ветвей, но не изменяют экспоненциальный характер сложности.

Затраты памяти состоят из хранения поля размером $N \times N$, текущего частичного решения, лучшего найденного решения и стека кадров для итеративного бэктрекинга. Поэтому пространственная сложность $O(N^2 + k)$, где k – количество квадратов в текущем решении.

Способ хранения частичных решений

В программе частичное решение хранится как набор уже размещенных на поле квадратов. Для этого используется структура Square с полями x, y и w типа int, где x – координата левого верхнего угла квадрата по оси X, y – координата левого верхнего угла по оси Y, а w – длина стороны квадрата.

Текущее частичное решение хранится в векторе currentSolution. При постановке нового квадрата он добавляется в конец этого вектора, а при откате удаляется. Таким образом, currentSolution всегда содержит текущее состояние рассматриваемой ветви поиска.

Лучшее найденное решение хранится отдельно в векторе bestSolution. Если найдено полное покрытие квадрата меньшим числом элементов, чем ранее, то текущее решение копируется в bestSolution.

Состояние поля хранится в одномерном массиве field, где каждая клетка принимает одно из двух значений: 0 – клетка свободна и 1 – клетка занята.

Одномерное хранение выбрано для уменьшения накладных расходов по памяти и ускорения доступа к клеткам. Для обращения к клетке с координатами (x, y) используется формула y * N + x.

Так как в работе реализован итеративный бэктрекинг, для хранения промежуточных состояний используется стек stackFrames. Каждый элемент стека – кадр поиска, содержащий: координаты первой свободной клетки на текущем уровне, следующий размер квадрата, который требуется проверить, строку, начиная с которой выполняется поиск следующей свободной клетки, квадрат, который был выполнен переход на данный уровень и признак наличия такого квадрата.

Таким образом, частичное решение в программе задается сразу несколькими структурами:

- field – текущее заполнение большого квадрата
- currentSolution – список квадратов текущей ветви
- stackFrames – стек состояний для итеративного поиска
- bestSolution – лучшее найденное решение

Использованные оптимизации алгоритма

1. Специальный случай для четных N

   Если размер исходного квадрата четный, то квадрат разбивается на 4 одинаковых квадрата размера N/2 × N/2.

2. Фиксация первого квадрата

   Для нечетных N первый квадрат всегда ставится в левый верхний угол. Это уменьшает число симметричных вариантов разбиения и сокращает дерево поиска.

3. Ограничение размера первого квадрата

   Для корневого состояния перебираются размеры квадрата от N - 1 до (N + 1)/2. Это позволяет исключить заведомо невыгодные варианты с слишком маленьким первым квадратом.

4. Построение жадной верхней границы

   Перед запуском полного перебора строится первое допустимое решение жадным способом: в первую свободную клетку ставится максимально возможный квадрат. Полученное решение задаёт начальную верхнюю границу bestCount, что позволяет раньше отсекать бесперспективные пути.

Тестирование и граничные случаи

1. Вход: 2

   Результат: квадрат разбивается на 4 квадрата

2. Вход: 3

   Результат: найдено минимальное разбиение на 6 квадратов

3. Вход: 4

   Результат: квадрат разбивается на 4 квадрата

4. Вход: 5

   Результат: найдено минимальное разбиение на 8 квадратов

5. Вход: 19

   Результат: найдено минимальное разбиение на 13 квадратов

Исследование

В ходе исследования была получена зависимость времени выполнения алгоритма от размера исходного квадрата N, представленная на рисунке 2.

Рисунок 2 – Зависимость времени выполнения от размера квадрата

По результатам исследования видно, что время выполнения сильно зависит от размера N. Для чётных значений время выполнения минимально за счет применения оптимизации. Для нечетных значений время работы заметно выше, так как в этих случаях используется итеративный бэктрекинг.

Таким образом, проведенное исследование показывает, что практическая скорость работы алгоритма сильно зависит от четности размера квадрата. Использованные оптимизации позволяют существенно сократить время работы, однако алгоритм сохраняет экспоненциальный характер роста в худшем случае.

Вывод

В ходе выполнения лабораторной работы был изучен метод поиска с возвратом и реализован алгоритм разбиения квадрата на минимальное число меньших квадратов.

В программе были реализованы структуры для хранения частичных решений, а также применены оптимизации, уменьшающие время работы алгоритма. Проведённое тестирование показало корректность работы программы на различных входных данных.

В рамках исследования была изучена зависимость времени выполнения алгоритма от размера квадрата. По результатам эксперимента установлено, что для четных значений N задача решается значительно быстрее, а для нечетных значений время работы возрастает существенно сильнее. Это подтверждает, что практическая эффективность бэктрекинга во многом зависит от используемых оптимизаций.